

Capesand Exploit Kit's Tools Seen in KurdishCoder Campaign

In November 2019, we published the analysis for new exploit kit Capesand, and further analysis of the indicators of compromise (IoCs) showed these samples were using obfuscation tools that made them virtually undetectable.

By: William Gamazo Sanchez, Joseph C Chen

December 04, 2019

Read time: 4 min (1180 words)

In November 2019, we [published a blog](#) analyzing an exploit kit we named Capesand that exploited Adobe Flash and Microsoft Internet Explorer flaws. During our analysis of the indicators of compromise (IoCs) in the deployed samples that were infecting the victim's machines, we noticed some interesting characteristics: notably that these samples were making use of obfuscation tools that made them virtually undetectable.

After some data collection we found more than 300 samples that correlate to the mentioned indicators that were recently very active our first detections occurred in August, with the campaign itself still ongoing (having occasional spikes in between). We saw a rising usage of tools that provide fully-undetectable obfuscation capabilities – signifying that the authors behind the samples designed their malware variants to be as stealthy as possible. We decided to name the potential campaign associated with these IoCs as “KurdishCoder”, based on the property name of an assembly module found in one of the samples.

We took a look at one of the samples captured from Capesand that was used to deploy the njRat malware – notably its main executable NotepadEx. We found that there were multiple layers of obfuscation using a combination of two tools: the .NET protectors ConfuserEx and Cassandra (CyaX). Both of these tools are used in combination to provide an array of fully undetectable capabilities to the deployed njRat malware variant.

Examining the Capesand samples

The simplified diagram taken from the previous blog shows the combination of ConfuserEx and Cassandra via the second layer of obfuscation protection, which involves the DLL CyaX_Sharp Assembly (both CyaX_Sharp and CyaX are part of the Cassandra protector).

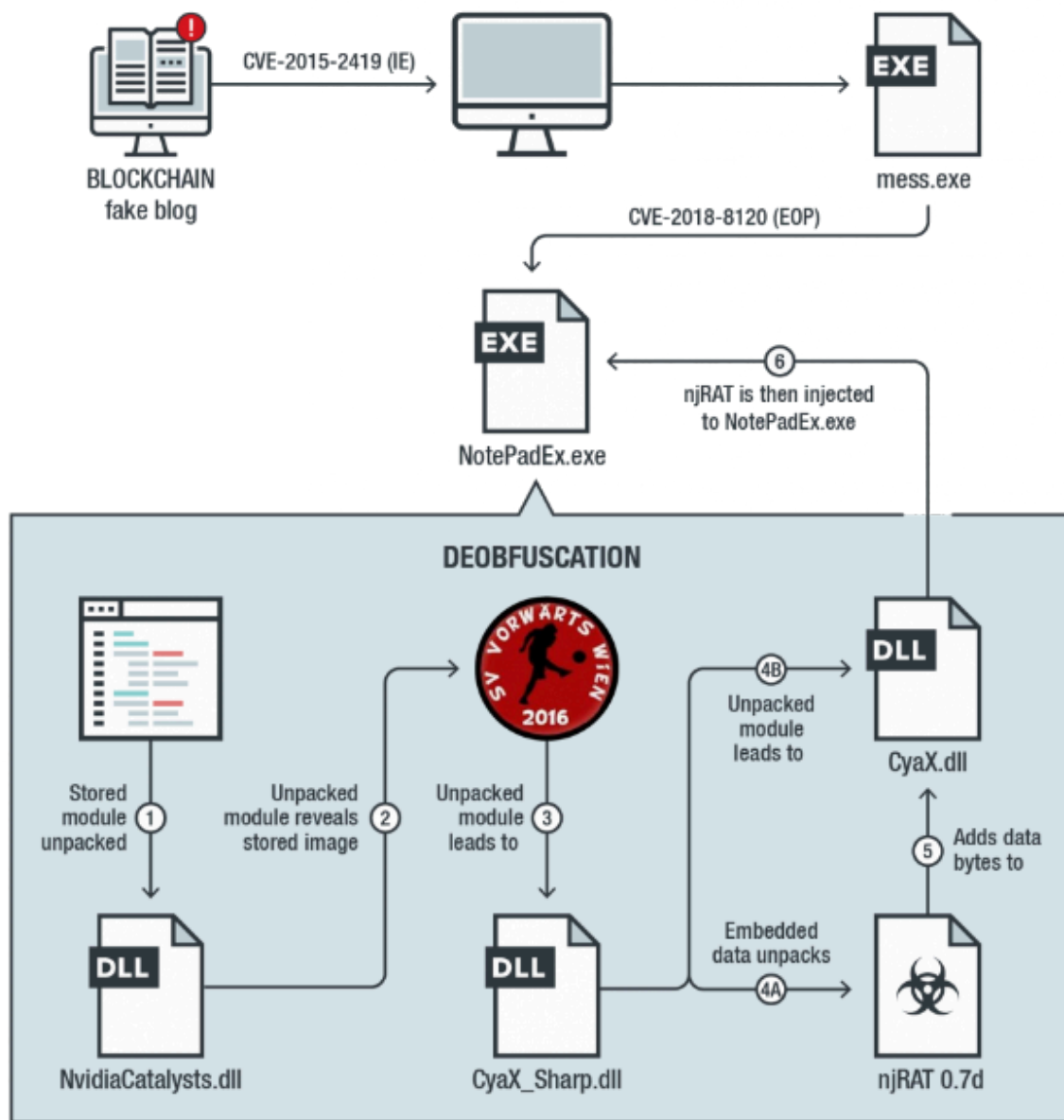


Figure 1. The infection chain for Capesand that also shows the obfuscation mechanisms

For this particular sample, CyaX_Sharp is obfuscated with a customized version of ConfuserEx. The following image shows an assembly module property that was generated for this sample.

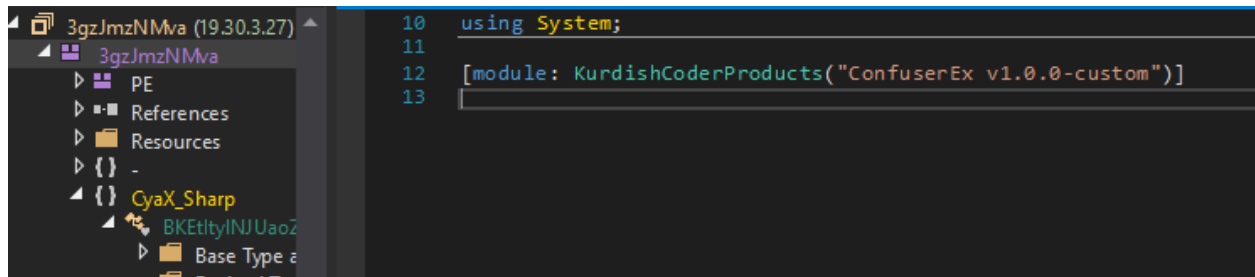


Figure 2. The generated assembly module property

The module's property name is "KurdishCoderProducts", with a value shown to be "ConfuserEx v1.0.0-custom". To understand where this value is coming from, let's take a look at the open-source ConfuserEx tool to see how the values are created. From there, we can establish a hypothesis as to their source.

A closer look at ConfuserEx's functions

While ConfuserEx is able to apply multiple transformations to the target binary, we are interested in two particular functions that we can use for correlation:

1. Source code building.

ConfuserEx is an open-source tool with multiple versions hosted on Github. By examining [one of the community-supported versions](#), we can see there is a tool to build ConfuserEx from the command line. This build command line has [a function](#) to update the final binary versions based on the last Git-tagged version. However, if ConfuserEx is built outside Git, the version update tool will just generate the value "version-custom" as shown below.

```

        catch {
            Console.WriteLine("error when executing git describe.");
        }
    }
    tag = tag ?? "v" + ver + "-custom";

    string template = Path.Combine(dir, "GlobalAssemblyInfo.Template.cs");
    string output = Path.Combine(dir, "GlobalAssemblyInfo.cs");

```

Figure 3. Code taken from a ConfuserEx version created outside GIT

Since the string "ConfuserEx v1.0.0-custom" is present in the module property: [module: KurdishCoderProducts("ConfuserEx v1.0.0-custom")], we can surmise that the version of ConfuserEx that was used for CyaX_Sharp was indeed built outside of Git.

• 2. Watermarking

When ConfuserEx performs its obfuscation routine, one of the operations creates a watermark – a unique identifier within the software--that is present in the final binary. The watermarking technique is implemented through the module attributes of the assembly. The following source code screenshot shows how this is implemented.

```
context.Logger.Debug("Watermarking...");
foreach (ModuleDefMD module in context.Modules) {
    TypeRef attrRef = module.CorLibTypes.GetTypeRef("System", "Attribute");
    var attrType = new TypeDefUser("", "ConfusedByAttribute", attrRef);
    module.Types.Add(attrType);
    marker.Mark(attrType, null);
}
```

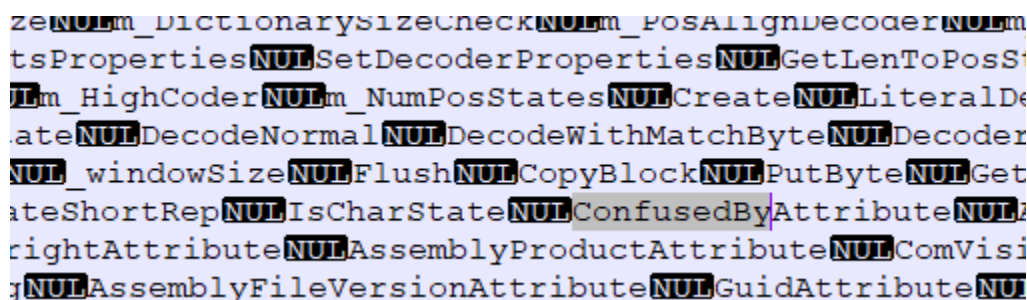
Figure 4. Code showing how the watermarking is performed via the module attributes of the assembly

From the previous code section, we can see the default attribute added by ConfuserEx is “ConfusedBy”. If we test it using a sample binary, the following is generated:

```
9
10 using System;
11
12 [module: ConfusedBy("ConfuserEx v1.0.0")]
13
```

Figure 5. Testing a sample binary using ConfuserEx

There are two important aspects to this attribute: first, it is hardcoded as a string constant and second, its value is presented in clear text in the final binary as the following image shows.



The image shows a list of assembly attributes in a binary format. The attributes include: DictionarySizeCheck, PosAlignDecoder, Properties, SetDecoderProperties, GetLenToPos, HighCoder, NumPosStates, Create, LiteralDe, DecodeNormal, DecodeWithMatchByte, Decoder, _windowSize, Flush, CopyBlock, PutByte, Get, ShortRep, IsCharState, ConfusedBy, Attribute, rightAttribute, AssemblyProductAttribute, ComVisi, AssemblyFileVersionAttribute, GuidAttribute, and another Attribute. The 'ConfusedBy' attribute is highlighted with a light blue background.

Figure 6. The default attribute shown in clear text in the final binary

We now have solid evidence that CyaX_Sharp was obfuscated using a modified version of ConfuserEx. Next, let’s look for the attribute indicator, starting with the CyaX assembly used in the NotepadEx attack.

Cassandra Protector: CyaX

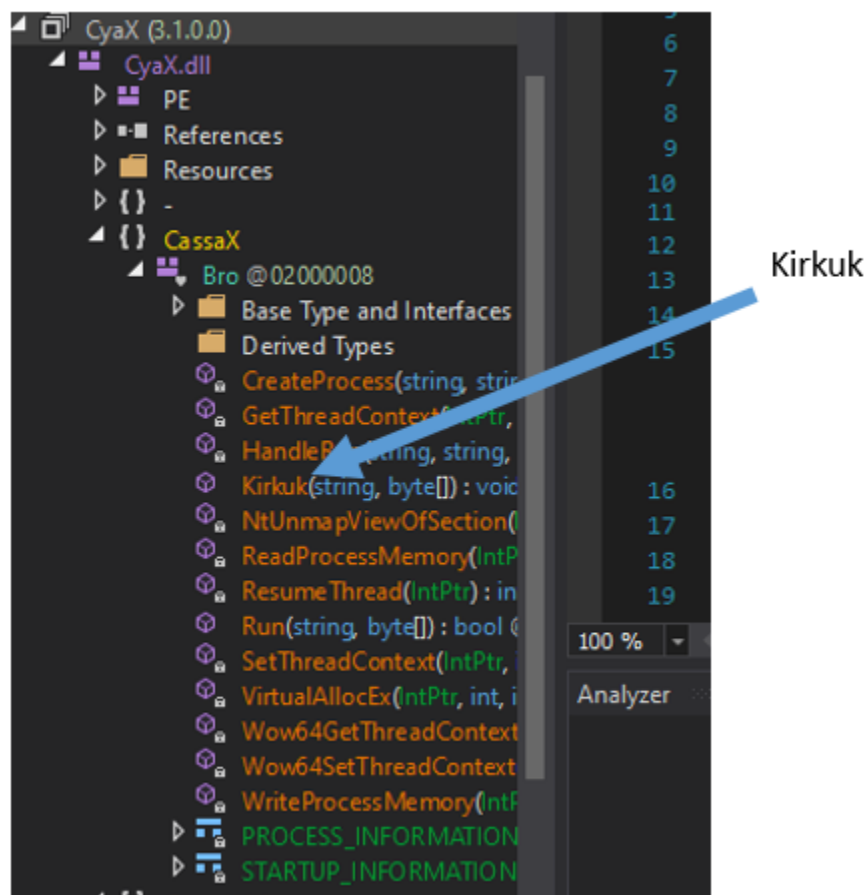


Figure 8. The modified method which was renamed to Kirkuk, which is also a name of a city in Iraq

What other payloads are using KurdishCoder?

Analysis of some of the captured samples reveals the different payloads being used (as shown in the table below). Note that this does not cover all the samples – it is possible that other payloads are being deployed as well.

Sample	Payload	KurdishCoderMain Sample	KurdishCoder_Cya X_Sharp	CyaX_method_rename
CustomIncreaseX	phoenix_key logger	Yes	Yes	Kirkuk
NotePadEx	Njrat	No	Yes	Kirkuk
QuickTranslation	Agent Tesla	Yes	Yes	Kurd
SandwichGenerator	Agent Tesla	Yes	Yes	Kirkuk
SimpleGame	Remcos	Yes	Yes	Kirkuk
AnimalGames	Hawkeye Reborn Keylogger	Yes	Yes	Kurd

Table 1. The different samples organized name, payload delivered and the fully-undetectable stages where the attribute “KurdishCoderProduction” is present

Cassandra Crypter

We think one of the possible sources of the customized ConfuserEx is the online service Cassandra Crypter, which offers two kinds of subscription plans: The Premium Plan and the Private Stub. The Premium Plan requires payment and works automatically, while the Private Stub requires the user to contact the support from the service for further personalization.

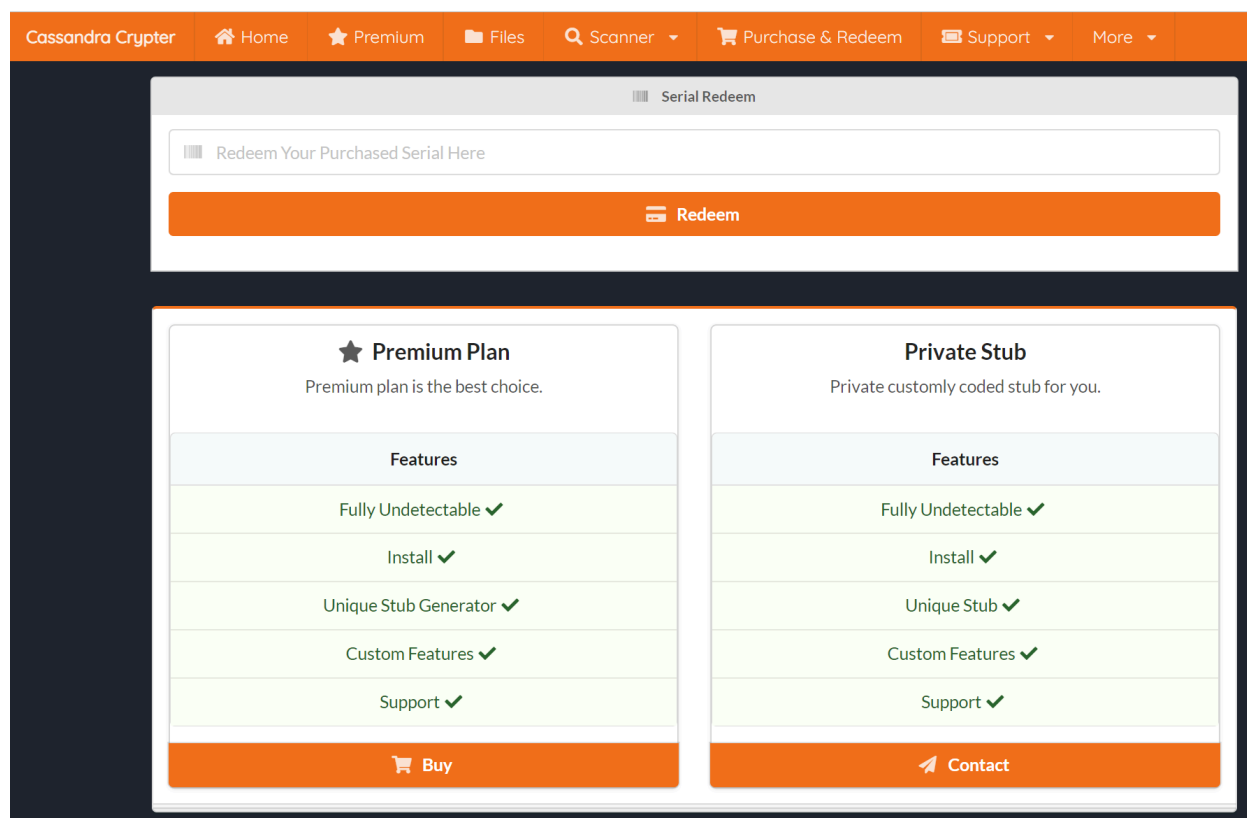


Figure 9. Cassandra Crypter's subscription plans

The combination of ConfuserEx and CyaX (Cassandra protector) seems to be unique and customized based on the indicators mentioned earlier. While we don't have definitive evidence that the use of these tools are part of a single campaign, we think the analyzed samples are related to a specific campaign. Note that the KurdishCoder indicator was spotted also by the Italian Computer Emergency Response Team - Pubblica Amministrazione (CERT-PA), which they [reported as a single incident](#).

As with Capesand, we will be monitoring the use of the tools mentioned in this blog entry for any future developments and updates.

Trend Micro Solutions

A proactive, multilayered approach to security is key against a wide range of threats— from the gateway, endpoints, networks, and servers. [Trend Micro™ endpoint security](#) has vulnerability protection that shields endpoints from identified and unknown vulnerability exploits even before patches are even deployed. Trend Micro's endpoint solutions such as [Trend Micro™ Smart Protection Suites](#) and [Worry-Free™ Business Security](#) protect end users and businesses from these threats by detecting and blocking malicious files and all related malicious URLs.

Indicators of Compromise (IoCs)

Indicator	Attribution	Trend Micro Pattern Detection
068d32a43191dc0164b600b85a1621be0154504fd477167422ff4a8fb3406d73	AnimalGames	Backdoor.MSIL.BLADABIN DI.QBR
07be156caac1157707ffe38266dc60abadc488226b4f41d67f23eac98dd917b0	CustomIncreaseX	Backdoor.MSIL.BLADABIN DI.QBR
b00cc9a4292fc5cc4ae5371ea1615ec6e49ebaf061dc4eccde84a6f96d95747c	NotePadEx	Backdoor.MSIL.BLADABIN DI.QBR
6755ce7a362ffecef805e4c54e1d5e201b6c6d561b997ebbd63a8d814ce6a53f	QuickTranslation	Backdoor.MSIL.BLADABIN DI.QBR
8ff11efc1109073fdc49be93e1d100992314fd68ecdff2ba986107602ce75089	SandwichGenerator	Backdoor.MSIL.BLADABIN DI.QBR
02f2369b58fbb2ba1df2c799b73842880a4874c32c1514a0d8956133be026ade	SimpleGame	Backdoor.Win32.REMCOS .USMANEAGDZ